"Wait, Cancel That": Commit Timing and Cancellation Semantics in an Oracle-Intent Model of Voice-Agent Actions

Qalab Hassnain Agha
Independent Researcher, Islamabad, Pakistan
ORCID: 0009-0006-6242-2761

30 October 2026

Abstract

Voice agents can begin tool execution before a user finishes speaking, but users can revise or cancel instructions while execution is in progress. A correct final database state does not establish that a cancellation prevented an earlier effect. We study this distinction using a deterministic, single-resource discrete-event model with oracle-parsed intent events. The model separates source intent from runtime receipt, reversible database writes from irreversible notification history, and prevention from compensation. We evaluate six execution-policy configurations over 1764 synthetic scenarios, producing 10584 runs. All five version-fenced configurations converge to the specified final database state under reliable delivery and successful compensation, yet their notification histories differ substantially. Among revision and cancellation scenarios, eager execution emits a subsequently superseded notification in 69.58% of cases, compared with 43.78% for preparation during the turn followed by a commit gate at its nominal end. On stable scenarios, that gated policy and eager execution both have 66.7 ms mean residual database-settlement latency, compared with 600.0 ms for preparation beginning at turn end. Removing version fencing reduces final-state correctness to 79.37%. These are finite-grid simulator results, not measurements of speech recognition, language models, human behaviour or production voice agents. The contribution is an auditable reference model and diagnostic decomposition of cancellation outcomes, with explicit assumptions and a path to end-to-end validation.

Keywords: voice agents; cancellation; speculative execution; asynchronous tools; version fencing; compensation; discrete-event simulation.

1. Introduction

Consider a user who says, "Send housekeeping to room 305 ... actually, 503." An agent that begins execution after recognizing the first room number can overlap tool preparation with the rest of the utterance. That overlap can reduce waiting, but it can also produce an obsolete dispatch. If the agent subsequently edits its database to show room 503, the final state may be correct even though somebody already received a room-305 notification. The user-visible statement "cancelled" collapses several operationally different outcomes.

The problem is relevant to systems that connect natural language with asynchronous work. Speech interruption concerns conversational output; task cancellation concerns scheduled or running work; compensation concerns an effect that has already happened. Their interfaces can coincide without their meanings coinciding. A runtime must track which instruction authorized a task, which effects have crossed a commit boundary, and which corrections it has actually received.

We isolate those execution semantics from recognition quality. The model receives a perfect sequence of structured intent events. It does not listen to audio, call a language model, infer a speaker's authority or predict turn endings. Consequently, the experiment cannot estimate the reliability of any voice model. Instead, it asks a systems question: given the same recognized intents and tool durations, how does commit timing change observable cancellation outcomes?

Three research questions guide the study. RQ1 asks whether final database correctness distinguishes prevention from recovery. RQ2 asks how preparation overlap and a fixed commit gate change latency and superseded-effect history. RQ3 asks what version fencing guarantees when corrections are delivered with delay.

The contributions are deliberately limited: (1) a traceable operational model with separate source and received intent versions; (2) metrics that distinguish known-stale execution, execution stale relative to an undelivered intent, retrospective supersession and final-state recovery; and (3) an exhaustive run over a declared finite parameter grid, accompanied by raw traces, a separate scorer and invariant tests. Neither

transactions nor compensation are claimed as new mechanisms. We make no claim to be the first study of cancellation in voice agents.

2. Related Work

2.1 Voice-agent execution and evaluation

Full-Duplex-Bench-v3 evaluates spoken systems using human audio with disfluencies and multi-step tool use [1]. Its scope includes recognition and interaction phenomena absent from our model. It is relevant to future validation of corrections, but our synthetic event traces cannot replace its audio evaluation.

Qwen-Audio-Agent describes foreground conversation with asynchronous background execution and explicitly separates speech interruption from task cancellation [2]. DuplexAgent-RSI also addresses cancellation, replacement and result delivery within a modular interaction harness [3]. These works make a broad novelty claim about "cancellation-aware voice agents" inappropriate. Our narrower focus is the distinction between state convergence and effect history in a small, inspectable scheduler model. We have not reimplemented or experimentally compared their systems.

VAmoS Bench evaluates complete voice agents against stateful customer-support scenarios, scoring action traces and backend changes [4]. It supplies a relevant precedent for checking what tools did rather than accepting an agent's spoken assertion. Our experiment adopts that evaluation principle in a substantially simpler environment without audio, SQL or model inference.

2.2 Speculative execution and recovery

Incremental dialogue processing already formalizes revocation and commitment of processing units [8]. These concepts predate modern voice-agent orchestration. Speculative Interaction Agents also separates early tool work from sensitive execution, supports modifying or removing in-flight calls, and holds sensitive tools for final confirmation [9]. These are close precedents for our preparation/commit distinction. Our study therefore does not claim a new general cancellation or speculative-execution mechanism. The narrower candidate contribution is an executable diagnostic comparison of temporal freshness, recovery and persistent

effect history; its publication-level novelty is not established by this targeted review.

Heartbeat Thinking investigates reasoning and tool execution while listening, allowing ongoing invocations to finish as updated input arrives [5]. Speculative Actions studies prediction and validation of future agent actions to overlap execution [6]. Those approaches motivate examination of early work and eventual commitment. Our model performs no speculative prediction: its initial target is supplied by an oracle and may subsequently be revised.

Compensating committed operations is established in transaction-processing literature, including Sagas [7]. A compensating operation provides application-level recovery rather than erasure of history. Our version-tagged compare-and-clear compensation is a small reference mechanism, not a replacement for distributed transaction protocols. We use it to expose the difference between removing a database assignment and retracting a notification that has already been observed.

The literature review is targeted rather than systematic. The cited official records and selected full-text sections were checked while preparing the manuscript. We also inspected two pinned public implementations with narrow extracted-function probes; those probes are contract evidence, not complete-agent evaluations. No submission-level novelty claim is made.

3. Operational Model

3.1 Scope and intent stream

Each scenario contains one mutable database resource: a room assignment. The assignment begins empty. An initial instruction targets room 305. A revision targets room 503; a second revision, when present, targets room 701. An explicit cancellation specifies an empty assignment. A non-cancelling interruption leaves the current instruction unchanged.

Every substantive intent has a monotonically increasing integer version. A source event changes the world's intended target at virtual time s. The runtime receives that same event at time r = s + d, where d is a fixed nonnegative delivery delay for subsequent events in the scenario. The initial intent is delivered at time zero. A non-cancelling interruption does not advance the version. The oracle supplies event type, target and version correctly.

We distinguish the latest source version W(t) from the latest received version R(t). A delayed correction creates an interval in which W(t) exceeds R(t). A runtime can fence against R(t), but cannot use a source event it has not yet received.

3.2 Preparation, commitment and notification

Preparation takes a declared virtual duration P and has no external side effects. Completing preparation permits an instantaneous database commit. The commit replaces the resource with the operation's version and target. A notification is scheduled N milliseconds after that commit. Once recorded, the notification remains in the observable history permanently.

Both the commit and notification optionally check that their operation version equals R(t). The check and state change form one atomic simulator event. This assumption is essential: a real implementation must enforce an equivalent check at the authoritative write or dispatch boundary. A check in application code followed by an unprotected remote write is not covered by this model.

The reference environment represents only one assignment and one notification per successful commit. It is not a three-domain benchmark of hotel booking, staff dispatch and financial cancellation. Multiple business labels would not create distinct tasks unless they introduced distinct state transitions and constraints.

3.3 Compensation

When a substantive update is received while the database is nonempty, the runtime schedules compensation after C milliseconds. Compensation captures the database version present at receipt. It clears the assignment only if that captured version still occupies the database when compensation executes. This compare-and-clear rule prevents a delayed recovery operation from deleting a newer assignment.

A revision also schedules preparation for the replacement assignment. A cancellation schedules no replacement. Notifications that were already emitted remain in history. Pending notifications can be suppressed by the version fence. Compensation is reliable in this experiment; its failure, retry and crash semantics are outside scope.

3.4 Event ordering and quiescence

All time is integer virtual milliseconds. At the same timestamp, events execute in this order: source updates, runtime receipts, compensation, database commits, notifications. Within each category, insertion order breaks ties. Thus a correction received exactly at a scheduled commit blocks that commit when fencing is enabled. Another ordering can change equality-boundary outcomes and should be reported explicitly in replications.

The queue is drained completely. There are no timeouts, dropped messages, duplicate deliveries, network failures, rate limits or random scheduling. Computational overlap is unlimited. This is a deterministic scheduling abstraction, not a performance measurement of concurrent infrastructure.

4. Policies and Elementary Properties

4.1 Policy definitions

Let E = 1000 ms be a fixed nominal boundary for the original turn. For a recognized operation received at r with preparation duration P, the baseline policies choose these commit times:

```
Wait: max(r, E) + P
Eager: r + P
Guarded(g): max(r + P, E + g)
```

Wait begins preparation no earlier than E. Eager prepares and commits immediately when ready. Guarded prepares early but delays commitment to a fixed gate E + g. We evaluate g in {0, 200, 500} ms. All five main configurations apply the same version checks and compensation rule.

The sixth configuration, Guarded-no-fence, uses g = 0 but removes both commit and notification version checks. It retains version-aware compensation. It is a diagnostic ablation, not a competitive state-of-the-art baseline. It isolates the effect of allowing scheduled obsolete work to complete.

The gate is fixed to the original turn, not a silence detector or a rolling debounce window. Later corrections do not move it. Corrections after E represent subsequent intent changes within the modeled interaction. Their inclusion is intentional, but it means that a longer grace period does not make the policy safe against

arbitrarily late revisions.

4.2 Received-version freshness

Proposition 1. With fencing enabled and atomic check-and-write, no database commit is stale relative to the latest received substantive intent at that instant.

Proof. Each substantive receipt replaces R with its newer version. A commit executes only if its version equals R at the atomic commit event. Therefore a commit whose version differs from R is rejected. Source updates that have not arrived do not change R, so the proposition says nothing about freshness relative to W. The same argument applies at the notification boundary. This is a direct property of the guard, not a learned capability. Its empirical verification checks implementation consistency with the definition.

4.3 Eventual final-state recovery

Proposition 2. Under the finite input stream, reliable delivery, finite preparation durations, successful compensation and fenced execution modeled here, the database eventually equals the final target.

Proof. After the last substantive receipt, no older operation can commit. If the final intent is a set operation, its finite preparation completes and the resulting commit is accepted. A previously scheduled compensation can clear only its captured older database version, not this final version. If the final intent is cancellation, no new operation is scheduled; any existing assignment is eventually cleared by its scheduled compensation, and older pending commits are blocked. Notifications do not alter the database. The proposition establishes convergence, not absence of prior external effects.

4.4 The visibility limit

Proposition 3. Received-version fencing alone cannot guarantee freshness against corrections whose delivery occurs after commitment.

Argument. Consider two executions with the same received history before commit time c. In one, no correction exists. In the other, a source correction occurs at $s < c$ but arrives at $r > c$. A deterministic runtime has identical information at c in both executions and therefore makes the same decision. If it commits in

the first, it also commits an operation stale relative to source intent in the second. A bounded grace period changes exposure but cannot rule out all such histories.

This is an information constraint, not an error that a stronger language model necessarily resolves. Explicit authorization handshakes, bounded-delay assumptions or delayed irreversible commitment would change the model and its guarantees.

4.5 Stable-case latency

For an unchanged instruction received at zero, residual settlement latency is measured after E. Wait has latency P; Guarded(g) has max(P - E, g); Eager has max(0, P - E). At g = 0, Guarded and Eager therefore have identical stable-case database latency in this abstraction. This equality depends on preparation having no side effects, no contention and no extra gate overhead.

5. Experimental Design

5.1 Fixed scenario grid

The scenario generator enumerates the following declared values:

Factor	Values
Intent pattern	Stable; non-cancelling interruption; revision; cancellation; two successive revisions
First change time	150, 450, 850, 1050, 1350, 1800 ms; absent for stable cases
Initial preparation P0	100, 500, 1200 ms
Revised preparation P1	100, 500, 1200 ms for revision and chain cases
Delivery delay d	0, 100, 350 ms for subsequent events
Notification delay N	0, 200 ms
Compensation delay C	0, 250 ms
Nominal original-turn boundary E	1000 ms

In a two-revision chain, the second change occurs 200 ms after the first and uses P0 for its preparation. Stable, interruption and cancellation cases use only one preparation-duration choice. Stable scenarios contain no change event. The resulting counts are 36 stable, 216 interruption, 648 revision, 216 cancellation and 648 chain scenarios, totaling 1764. Six policy configurations yield 10584 runs.

Some factor combinations are behaviourally equivalent: for example, delivery delay does not affect a stable scenario. They remain in the grid to keep enumeration transparent. This affects weighting, so aggregate percentages are properties of this grid, not estimates of user behaviour. Kind-specific results accompany aggregate results. No held-out split is claimed because no model is trained and no policy parameter is fitted. The study was not preregistered.

5.2 Outcome definitions

Final-state correctness means that the database target at quiescence equals the last substantive source target, including an empty target for cancellation.

Known-stale commit incidence records whether a run contains any commit whose version differs from R at commitment. **World-stale commit incidence** compares the version with W instead. The latter can occur even when a policy respects every intent it has received.

Superseded commit or notification incidence is retrospective: among revision, cancellation and chain scenarios, it records whether any committed or notified version differs from the final substantive version. An effect can be legitimate when emitted and become superseded later. This metric must not be relabeled an unauthorized-action rate or an attack-success rate.

Cancellation prevention means that no database commit occurred anywhere in a cancellation scenario. **Cancellation recovery** means that the final database is empty. Their distinction is intentional: editing back an assignment does not establish prevention.

Residual settlement latency is the time until the database becomes continuously equal to the final target through quiescence, minus H = max(E, last substantive source time), clipped at zero. Incorrect final runs have no settlement latency and are excluded from successful-run means. The stable-case mean avoids that success-selection issue. Latency excludes audio generation, notification delivery and any acknowledgment to the user. It must not be interpreted as end-to-end conversational response latency.

5.3 Reproducibility and validation

The Python implementation uses only the standard library. It emits a JSONL scenario set, CSV run outcomes, compressed JSONL event traces, aggregate and stratified results, and a manifest with source and artifact SHA-256 hashes. The manifest records the interpreter version. The simulator uses no random seed because it makes no random draws.

Twenty-eight unit and invariant tests cover analytic stable latency, cancellation before and after commit, equal-time ordering, delayed delivery, stale writes without fencing, suppression of pending notifications, permanence of emitted notifications, interruption preservation, compensation races, dataset-manifest safety, metadata ingestion and the full-grid convergence property. A separate audit program recomputes the principal Boolean metrics and settlement latency from saved traces rather than trusting the CSV metrics.

6. Results

6.1 Aggregate outcomes

Policy	Final correct	Known-stale commit	World-stale commit	Superseded notification*	Stable latency (ms)
wait	100.00%	0.00%	10.88%	23.94%	600.0
eager	100.00%	0.00%	12.24%	69.58%	66.7
guarded_0	100.00%	0.00%	9.52%	43.78%	66.7
guarded_200	100.00%	0.00%	8.84%	34.52%	200.0
guarded_500	100.00%	0.00%	7.48%	20.63%	500.0
guarded_no_fence	79.37%	55.10%	62.13%	100.00%	66.7

Each policy: 1,764 runs. *Changed-scenario denominator: 1,512 per policy. Stable latency: 36 stable scenarios per policy.

Every fenced configuration reaches 100% final-state correctness in this finite grid. That result is expected under Proposition 2 and should be read as an implementation check under favourable recovery assumptions. It does not establish 100% reliability for real agents.

Notification histories distinguish these otherwise identical final-state outcomes. Eager execution has a 69.58% superseded-notification incidence on changed scenarios. Guarded(0) reduces it to 43.78%, while

the stable-case mean residual settlement latency remains 66.7 ms for both. Wait reduces superseded notifications further but incurs 600.0 ms stable-case mean latency. Guarded(200) and Guarded(500) reduce retrospective supersession further while increasing stable-case latency. No policy dominates under every weighting of timeliness and historical effects.

6.2 Prevention differs from recovery

Policy	Cancellation scenarios	Prevented any commit	Final empty database
wait	216	75.93%	100.00%
eager	216	27.78%	100.00%
guarded_0	216	50.00%	100.00%
guarded_200	216	61.11%	100.00%
guarded_500	216	77.78%	100.00%
guarded_no_fence	216	0.00%	50.00%

All fenced policies recover the final empty state in cancellation cases. Their prevention rates differ because some have already committed before receiving cancellation. Thus a metric restricted to final database correctness would erase a major distinction among policies. An emitted notification survives compensation even when the database subsequently becomes empty.

6.3 Delayed corrections

Policy	d = 0 ms	d = 100 ms	d = 350 ms
wait	0.00%	14.29%	23.81%
eager	0.00%	19.05%	23.81%
guarded_0	0.00%	9.52%	23.81%
guarded_500	0.00%	4.76%	21.43%

World-stale commit incidence among changed scenarios: 504 per policy and delivery-delay setting.

With zero delivery delay, fenced policies produce no world-stale commits on changed scenarios. Positive delay creates exposure. Longer gating reduces some exposure but does not eliminate it. Timing values are coarse and deliberately chosen; these rates do not imply that actual message delivery follows this distribution.

6.4 Fencing ablation

Guarded-no-fence reaches 79.37% final correctness overall. Its correctness is 91.36% for one-revision cases, 50.00% for cancellation and 69.14% for revision chains; stable and interruption cases remain correct. A common failure is that a slow initial preparation commits after a faster replacement. Another is a pending old operation committing after cancellation was processed while the database was still empty. No compensation was scheduled at that receipt because no effect existed yet.

This ablation is intentionally weaker than a correctly implemented cancellable runtime. It diagnoses why ignoring late completions is necessary; it does not show superiority over published voice-agent systems.

6.5 Controlled authoritative-boundary validation

We added a separate local asynchronous testbed with eight deterministic cases. Each case injects a correction before tool entry, after an application authorization check, after database commit but before notification, or after notification. We compare an application-check-only implementation with one that checks the current intent version atomically at both database and notification boundaries. The testbed uses SQLite and controlled coroutine barriers; it contains no speech model, remote service, or external notification delivery.

The cases reproduce the simulator's metric distinction. An application check followed by an unguarded write accepts an obsolete write when cancellation arrives after the check. An authoritative version predicate rejects that write. When a write has occurred, compensation can restore the database while prevention remains false. When a notification has been accepted, the database can be empty at quiescence while the notification remains in history. These are executable contract examples, not failure-rate estimates or tests of the upstream voice-agent implementations.

6.5 Worked counterexample

Take an initial preparation of 1200 ms, a revision at 150 ms, a revised preparation of 100 ms, zero delivery delay and Guarded(0). The revised operation is ready at 250 ms and waits until 1000 ms to commit. The original operation is ready at 1200 ms. With fencing, that old completion is rejected because version 2 has already been received. Without fencing, it replaces room 503 with room 305 at 1200 ms. Both histories are

available as event traces, and the corresponding behaviour is checked by tests.

6.6 Open-data interaction stress test

We downloaded the four public Full-Duplex-Bench v1.5 categories and processed 498 metadata records outside the repository [10]. The source categories comprise 200 user-interruption, 99 user-backchannel, 100 talking-to-other and 199 background-speech records. We deliberately did not reinterpret these labels as cancellation ground truth. The 200 user-interruption records were marked unclassified, and the remaining 298 records were retained as non-cancellation-overlap controls. Consequently, this release reports zero explicit cancellation records and zero cancellation-policy evaluations from the open audio. The result validates the metadata/replay boundary and exposes the need for a separate audio-and-task annotation study; it does not validate the cancellation policies on real speech.

Source category	Records	Treatment	Evidence status
User interruption	200	Unclassified	Overlap label is not cancellation ground truth
User backchannel	99	Non-cancellation-overlap control	Negative control only
Talking to other	100	Non-cancellation-overlap control	Negative control only
Background speech	199	Non-cancellation-overlap control	Negative control only
Total	**498**		**0 explicit cancellation records**

7. Discussion

The results support a diagnostic claim rather than a model-performance claim. Final-state recovery, prevention and historical effects must be reported separately. A runtime that always converges can still execute work that users later supersede. Conversely, retrospectively superseded work is not necessarily a violation of the instruction available when it ran. The separate world-stale metric makes that temporal distinction explicit.

Preparation overlap is attractive when preparation is read-only, reusable and cheap enough not to cause contention. The stable-case latency equation quantifies that idealized opportunity. It does not justify over-

lapping an irreversible action with incomplete authorization. Where preparation consumes scarce resources, creates reservations or triggers billing, its side effects must enter the model rather than being treated as free.

The model also suggests precise runtime contracts. Operations should carry intent versions to the authoritative commit and notification boundaries. A cancellation acknowledgment should distinguish "prevented," "database effect compensated," "notification already emitted" and "outcome unknown." Those are suggested interface semantics; this experiment does not measure user comprehension or implement spoken acknowledgments.

The absence of model inference is both useful and limiting. It isolates execution bugs without confounding them with speech transcription. It also removes a likely dominant source of real failure: misclassifying an interruption as cancellation, missing a correction or attributing another speaker's instruction to the user. A reliable execution layer cannot fix the wrong intent stream.

8. Limitations and End-to-End Validation

Construct validity. The task is a single mutable assignment and immutable notification log. It omits multi-resource dependencies, booking constraints, dispatch acceptance, payment settlement and genuinely irreversible physical work. Compensation is successful and has simple semantics. A database field returning to its initial value need not restore business reality.

Timing validity. Virtual durations are selected experimental factors rather than fitted service-time distributions. Preparation has unlimited concurrency and no cost model. The fixed original-turn gate is not a production endpoint detector. Equality-boundary priority is favourable to received corrections and may differ from a real event loop.

Recognition and authorization. Intent parsing is perfect, event ordering is preserved and each revision has a known version. No microphone, ASR system, LLM, human participant or external service is involved. Neither the rates nor the theoretical properties validate any commercial or open-source voice model.

Recovery and infrastructure. Delivery is reliable, compensation never fails, and there are no duplicate requests or retries. Idempotency, crash recovery, persistent outboxes, reordered messages, remote write uncertainty and resource locking are not tested. The atomic fence is an assumption, not a demonstrated

distributed protocol.

External validity and novelty. The aggregate weights are synthetic. Scenario variants are not independent human observations; significance tests or population confidence intervals would be misleading. Existing voice harnesses already address cancellation, replacement, speculative execution and confirmation. Our upstream code probes are narrow and do not establish a defect or priority claim. The controlled runtime testbed demonstrates a contract and metric distinction, but a stronger publication still needs complete-system adapters and real tool/notification measurements.

The open-data stress test has additional limits. Full-Duplex-Bench v1.5 contains overlap categories rather than task-level cancellation labels, so the conservative annotation produces no cancellation ground truth. Audio endpoint ambiguity, dataset-specific speaking styles and the upstream license terms also limit generalization. The downloaded source archives remain outside the public repository.

A follow-up should freeze a versioned scenario specification, collect consented human speech containing corrections and non-cancelling interruptions, and annotate source intent and correction timing independently. It should compare cascaded and direct-audio systems on identical tool interfaces, record both oracle and predicted intent streams, and introduce delivery failures and retries. Measurements should include actual audio endpoints, database effects, notification effects, cancellations acknowledged incorrectly and compute cost. Those experiments have not been conducted here.

9. Conclusion

In a deterministic oracle-intent model, final database correctness is insufficient to characterize voice-agent cancellation. Version fencing prevents commits stale relative to received intent and, with reliable compensation, ensures convergence in the modeled setting. It does not prevent effects stale relative to corrections still in transit, nor erase notifications already emitted. The controlled boundary test shows why these outcomes require separate checks and acknowledgment states. Early preparation with a commit gate changes the latency and supersession trade-off without requiring a new language model, but it does not dominate the added debounce baseline. The released artifact makes these distinctions executable and auditable. Claims about real speech systems and publication-level novelty remain open to further evidence.

Research Integrity, Assistance and Availability

This manuscript, experiment design, simulator, tests and literature notes were substantially developed with an AI coding assistant at the author's request. The numerical results come from executing the included simulator; no speech-model scores, participant counts or production measurements have been invented. The author has completed an independent review of the manuscript, implementation, proofs and references. The author confirms the listed name and independent-researcher affiliation; the ORCID is listed above.

No newly collected human-subject data, private hotel records, clinical information, credentials or production endpoints were used. The open-data stress test processed public human speech from Full-Duplex-Bench v1.5 under its source terms; source audio and transcripts are not redistributed in this repository. All room identifiers and synthetic events are fictional. Code, generated scenarios, evaluation scripts, aggregate results and manuscript sources are available at https://github.com/qalab-e-hassnain/cancelbench. No DOI or arXiv identifier is claimed. Software is offered under the repository's MIT license; the manuscript's publication license remains for the author to select.

References

[1] Guan-Ting Lin, Chen Chen, Zhehuai Chen, and Hung-yi Lee. Full-Duplex-Bench-v3: Benchmarking Tool Use for Full-Duplex Voice Agents Under Real-World Disfluency. arXiv:2604.04847, 2026. https://arxiv.org/abs/2604.04

[2] Chong Deng, Yunjie Ji, Yuxiang Kong, Xiangang Li, Xu Li, Binbin Zhang, Haina Zhu, and Jianheng Zhuo. Qwen-Audio-Agent Technical Report. arXiv:2609.25195, 2026. https://arxiv.org/abs/2609.25195

[3] Yingda Shen, Yuxiang Wang, Kunyu Feng, Qinke Ni, Jiaqi Li, Minghao Hsu, Junan Zhang, Dekun Chen, Yutong Bian, and Zhizheng Wu. DuplexAgent-RSI: Recursive Harness Improvement for Full-Duplex Voice Agent Collaboration. arXiv:2610.11299, 2026. https://arxiv.org/abs/2610.11299

[4] Joshua Meyer, Sahar Shayegan, Ritiz Tambi, Ali Khan, Sun Kim, Victor Shih, Mehdi Jamei, and Andi Partovi. VAmoS Bench: Voice Agent Simulation Bench. arXiv:2607.27453, 2026. https://arxiv.org/abs/2607.27453

[5] Zhenyu Zhang, Kewei Fu, Yongqi Tong, Chenzhi Zheng, Wei Wu, Jiang-Ming Yang, and Jianshe Li. Heartbeat Thinking: Speculative Reasoning and Tool Execution While Listening in Realtime Voice Agents.

arXiv:2610.26233, 2026. https://arxiv.org/abs/2610.26233

[6] Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, and Tianyi Peng. Speculative Actions: A Lossless Framework for Faster Agentic Systems. arXiv:2510.04371, version 2, 2026. https://arxiv.org/abs/2510.04371v2

[7] Hector Garcia-Molina and Kenneth Salem. Sagas. ACM SIGMOD, 1987. Author technical-report version: https://www.cs.princeton.edu/techreports/1987/070.pdf

[8] David Schlangen and Gabriel Skantze. A General, Abstract Model of Incremental Dialogue Processing. Dialogue & Discourse 2(1), 83–111, 2011. https://clp.ling.uni-potsdam.de/publications/Schlangen-2011.pdf

[9] Coleman Hooper, Minwoo Kang, Suhong Moon, Nicholas Lee, Eric Wen, John Wawrzynek, Michael W. Mahoney, Yakun Sophia Shao, Amir Gholami, and Kurt Keutzer. Speculative Interaction Agents: Building Real-Time Agents with Asynchronous I/O and Speculative Tool Calling. arXiv:2605.13360, 2026. https://arxiv.org/abs/2605.13360

[10] Guan-Ting Lin, Jiachen Lian, Tingle Li, Qirui Wang, Gopala Anumanchipalli, Alexander H. Liu, and Hung-yi Lee. Full-Duplex-Bench: A Benchmark to Evaluate Full-Duplex Spoken Dialogue Models on Turn-Taking Capabilities. v1.5 dataset release, 2025. https://github.com/DanielLin94144/Full-Duplex-Bench/tree/main/v1_v1.5

Appendix A. Reproduction

Run from the repository root using Python 3.9 or newer:

```
PYTHONPATH=src python3 -m unittest discover -s tests -v
PYTHONPATH=src python3 -m cancelbench.run --output results
PYTHONPATH=src python3 -m cancelbench.audit --results results
python3 scripts/build_paper.py
```

The build script inserts measured tables into the manuscript template and produces Markdown, HTML and LaTeX versions. The LaTeX source is provided for compilation with a local TeX installation. The manifest identifies the exact simulator source and experiment artifacts. Repeat execution should produce

byte-identical experiment artifacts on the same interpreter; the recorded interpreter version intentionally distinguishes environments.

Appendix B. Minimal Execution Pseudocode

```
on substantive receipt(version, target):
    latest_received_version = version
    if database is nonempty:
        schedule compare-and-clear(database.version) after C
    if target is not cancellation:
        schedule prepared operation according to policy


on scheduled commit(operation):
    atomically:
        reject unless operation.version == latest_received_version
        database = (operation.version, operation.target)
    schedule notification after N


on notification(operation):
    atomically:
        reject unless operation.version == latest_received_version
        append operation to immutable notification history
```

Guarded-no-fence omits the two version checks. Scheduled obsolete preparations may still finish in fenced configurations, but their commits are rejected. Cancellation therefore invalidates authorization for pending effects rather than necessarily preempting the preparation computation itself.